OpenJDK distribution I've been using locally so far. Then you need to copy the JAR file of Catalog Service into the image itself. Finally, declare that the entry point for the container execution is the command to run the application on the JRE.

Open your Catalog Service project (catalog-service), and create an empty file called Dockerfile (with no extension) in the root folder. That file will contain the recipe for containerizing your application.

**Listing 6.2 Dockerfile for describing the Catalog Service image**

```
FROM eclipse-temurin:17                                  ❶
WORKDIR workspace                                        ❷
ARG JAR_FILE=build/libs/*.jar                            ❸
COPY ${JAR_FILE} catalog-service.jar                     ❹
ENTRYPOINT ["java", "-jar", "catalog-service.jar"]       ❺
```

❶ Ubuntu base image with the Eclipse Temurin distribution of the JRE pre-installed

❷ Changes the current working directory to "workspace"

❸ Builds argument specifying the location of the application JAR file in your project

❹ Copies the application JAR file from the local machine into the image

❺ Sets the container entry point to run the application

This Dockerfile declares a `JAR_FILE` argument that can be specified when creating the image with the `docker build` command.

Before moving on, you need to build the JAR artifact for the Catalog Service application. Open a Terminal window and navigate to the Catalog Service project's root folder. First, build the JAR artifact:

```
$ ./gradlew clean bootJar
```

By default, the Dockerfile script will copy the application's JAR file from the location path used by Gradle: `build/libs/`. So if you're using